EMBEDDED ML · COMPUTER VISION · FLIGHT CONTROL

I built a drone that follows one specific person.

The printed circuit board IS the airframe.

No separate frame — the motors clamp into rings routed through the arms.

92 mm · 66 g · ESP32-S3

292 tests · MIT licensed

> It has never flown — the boards are not fabricated yet. Everything here is measured on the bench or in simulation.

THE PROBLEM

Following a person is not detection.

A detector finds every person in frame. Following needs the RIGHT one, held across occlusion, motion blur and bad angles.

So identity is two signals with different jobs:

FACE acquires and re-confirms. Strict.

ArcFace 512-d, threshold 0.36.

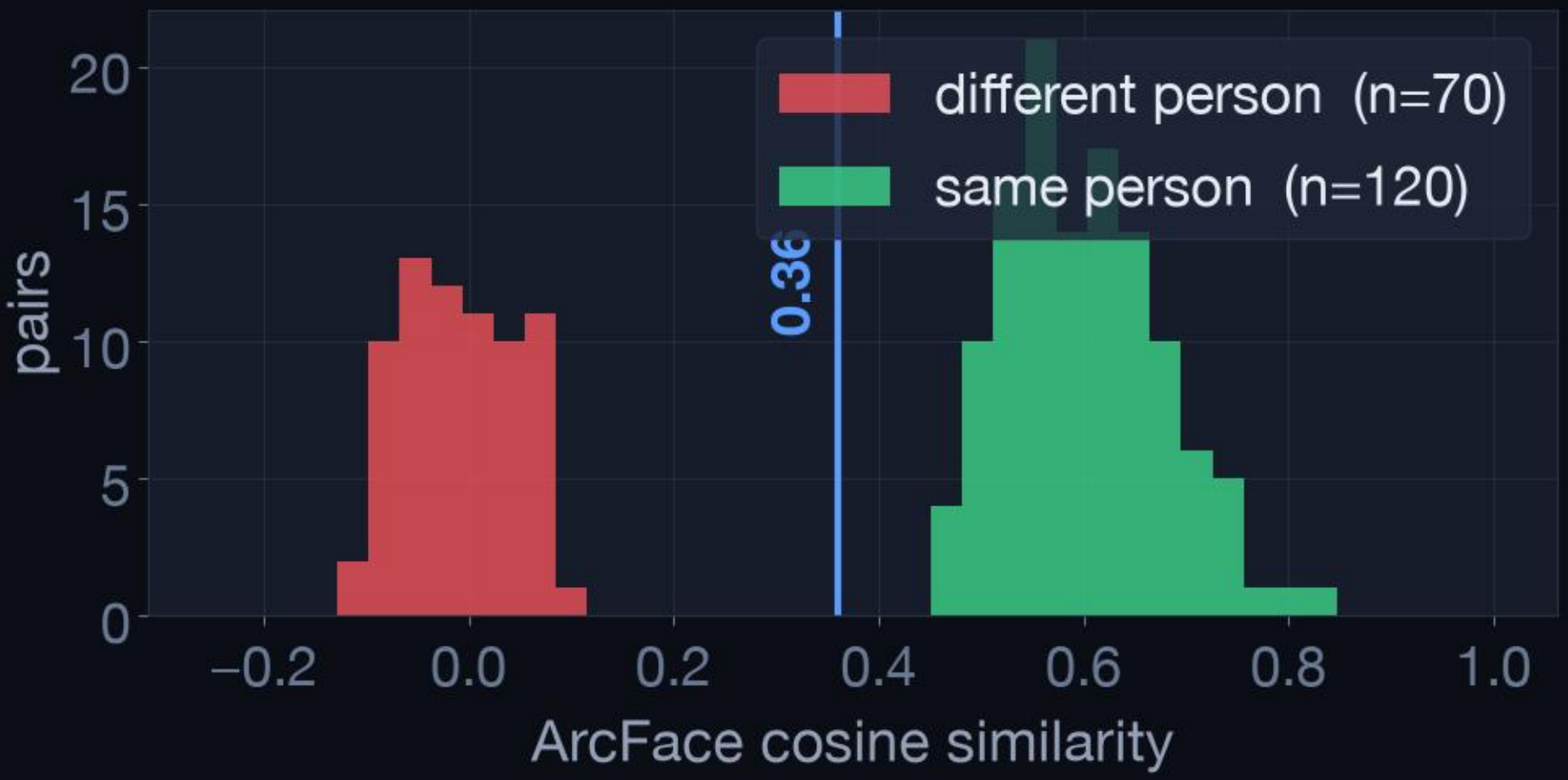
BODY sustains between face frames.

Re-ID 768-d — and it has to, because only 26% of frames carry a usable face.

A readable face BELOW 0.15 is evidence AGAINST. Without that veto, two people in similar clothing swapped identity for good.

EVALUATION — DOES IT ACTUALLY SEPARATE?

I measured it instead of trusting it.



Separation +0.350 · no overlap · AUC 1.000

0 false accepts in 70 · 0 false rejects in 120

Only 5 identities, so the 95% CI on the false-accept rate is [0, 5.2%]. That is a statement about SAMPLE SIZE, not the model.

THE OPERATING POINT

The two errors are not equally bad.

FALSE REJECT — miss one frame.
Costs nothing. Body re-ID carries the lock, and acquisition needs 5 consecutive hits.

FALSE ACCEPT — lock onto a stranger.
The drone follows the wrong person, and the appearance model then REINFORCES the error.

So the threshold buys a negligible false-accept rate using false rejects, which are nearly free. 0.36 sits well above the 0.108 where accepts first appear — deliberately conservative.

A threshold is a DECISION. It should be argued from measured error costs — which is the part most projects skip.

WHAT THE EVALUATION CAUGHT

My own dataset was mislabelled.

Two identities enrolled: 'Paing' and 'alex'.

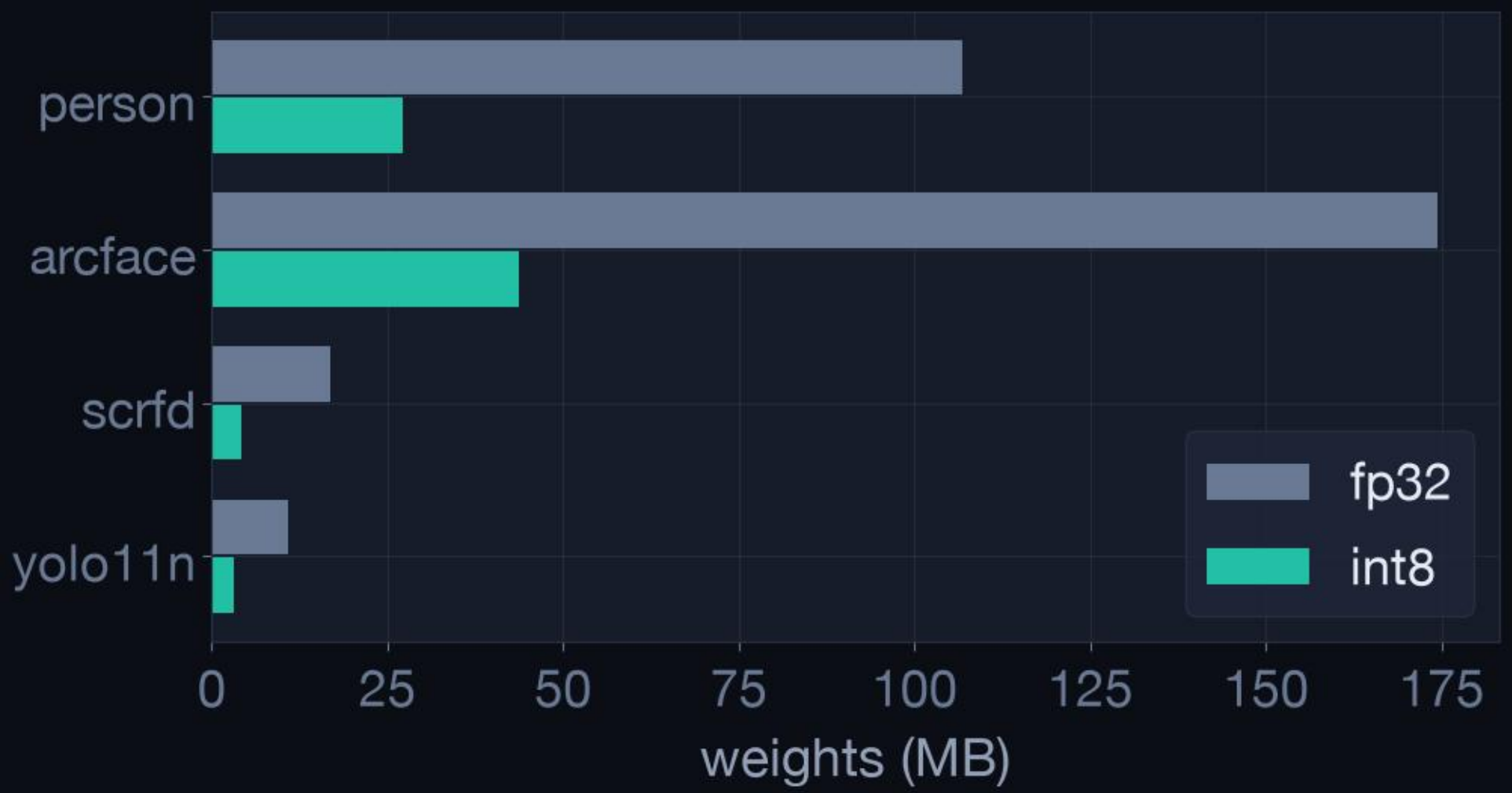


A stranger cannot resemble you more than you do. 'alex' was a placeholder enrolled from my own webcam and never replaced.

Every 'different person' pair was really the same person. The first run reported a confident AUC of 0.83 — fiction. Now it self-checks.

DEPLOYMENT — INT8 QUANTISATION

4x smaller.
And slower.



309 MB -> 78 MB · latency 0.89x (WORSE)

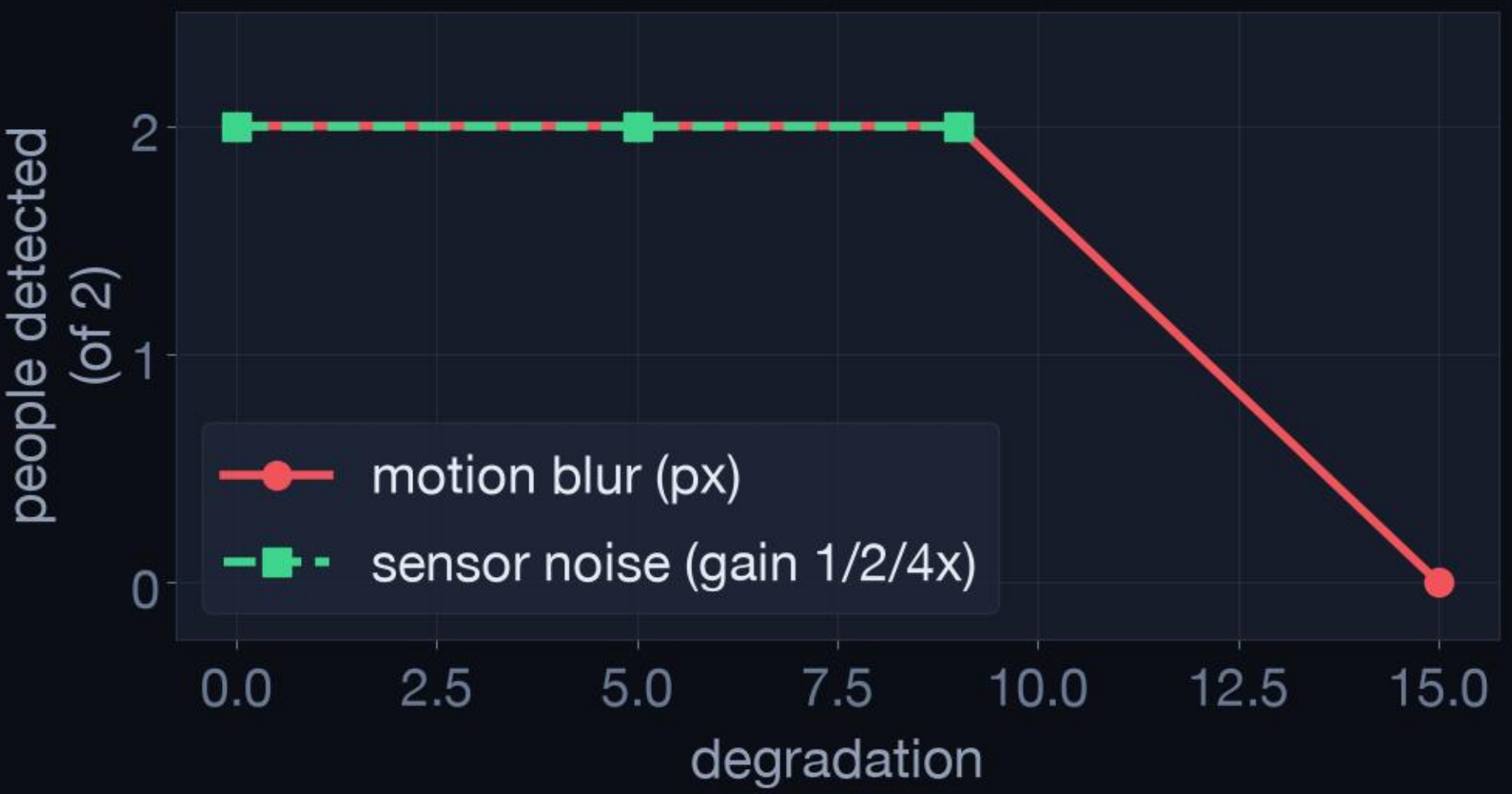
output agreement 0.976 – 0.999

CoreML runs fp16 on the Neural Engine faster than the CPU runs INT8. So here it is a memory win and a latency LOSS. It would win on a Pi.

THE EXPERIMENT THAT CHANGED THE HARDWARE

Noise is fine.

Blur is fatal.



YOLO keys on edges. Noise leaves them intact.

Blur destroys them. At 15 px of blur: zero people.

So the camera now PREFERS a dark, noisy, short-exposure frame. The old config capped gain to avoid noise — exactly backwards.

CONTROL — THE RESULT THAT MATTERS

The loop closes through the airframe.



Pitching to chase motion at the camera, which moves where the subject appears. That path has gain 34/26 = 1.31 — above one.

Above unity, the correction overshoots the error that caused it. Uncompensated it rings at ONE frame — and the real RTT is 20 ms.

WHAT IS NOT DONE

It has never flown.

The boards are not fabricated. The PID gains are scaled from typical values and have never touched this airframe. The stability result above is a simulation with ESTIMATED mass properties.

Also true:

- nothing here was TRAINED — the models are pretrained; the work is evaluation and deploymen
- the two-person discrimination test has never actually been run on two real people
- the face models are non-commercial research us

I would rather you know the limits than discover them. Everything claimed above is measured. This slide is what is not.

THE STACK

Built end to end.

PCB generated from a Python netlist, with a 7-gate check that must pass before fab

FIRMWARE ESP32-S3 from the IMU registers up: Madgwick, cascaded PID, mixer, watchdog

VISION ONNX Runtime + CoreML, no torch: detect, track, re-identify, servo

PROTOCOL generated from ONE source, so the two ends cannot drift. Checked in CI.

207 Python + 85 native C++ tests

Full write-up, every measurement, the code:
github.com/Pai-88/pursuit-drone